Universidad Nacional de Córdoba
Facultad de Ciencias Exactas, Físicas y Naturales
Ingeniería en Computación

Redes de Computadoras

Monografía

MQTT y Protocol Buffers

Un stack de comunicación en tiempo real para IoT, robótica y mensajería con servicios en la nube

Equipo
WireGuardians

Viberti, Benjamin[1]	DNI 46224179
Espinoza Sutta, Aaron Alejandro[1]	DNI 96009173
Cleri, Juan Ignacio[1]	DNI 46452662
Pineda, Juan Ignacio[1]	DNI 45591343
Grafión, Atilio Leonel[1]	DNI 43940195
Badenes, Tomás[1]	DNI 44785038
Oviedo, Ignacio Nicolás[1]	DNI 44169239
Mendez, Jorge Nicolas[1]	DNI 41301342

Docente
Oliva Cuneo, Facundo Nicolas

[1]{b.viberti,aaron.espinoza_4500,ignacio.cleri,juan.ignacio.pineda,atilio.grafion,tomasbadenes,
ignacio.oviedo.239,jorge.mendez}@mi.unc.edu.ar

BORRADOR

27 de octubre de 2026

Índice

Introducción **3**
1. Objetivos 4
2. Alcance 4
Marco teórico **5**
1. Modelo publicador/suscriptor 5
2. MQTT 5
3. Protocol Buffers 5
4. Stack MQTT y Protocol Buffers 5
5. Comparación con alternativas 5
   5.1. MQTT, HTTP y CoAP 5
   5.2. Protocol Buffers y JSON 5
Desarrollo **6**
1. MQTT sobre TCP y TLS 6
   1.1. Conexión persistente y keep-alive 6
   1.2. Cifrado con TLS 6
   1.3. QoS MQTT sobre la confiabilidad de TCP 6
2. Capa de red 6
   2.1. NAT y firewalls 6
   2.2. Ruteo interno 6
   2.3. Ruteo externo 6
   2.4. Convergencia y conexiones de larga duración 6
3. Ruteo por tópicos y ruteo IP 6
4. Casos de uso 6
5. Diseño experimental 6
   5.1. Plataforma común 6
   5.2. Experimento 1: MQTT contra HTTPS para telemetría 7
   5.3. Experimento 2: sincronización de osciladores acoplados 7
Conclusiones **8**
Bibliografía **9**
Anexos **10**
A. Mensajes del experimento de sincronización 10

BORRADOR

Introducción

Gran parte de la Internet comercial se apoya en el modelo de solicitud y respuesta de HTTP: un cliente consulta a un servidor cuando necesita un dato y recibe la respuesta, que suele venir codificada en formatos de texto como HTML, JSON o XML. Este modelo está pensado para intercambios puntuales, que el cliente inicia a demanda, y no para flujos de datos continuos ni para que el servidor tome la iniciativa.

Un número creciente de sistemas no encaja en ese modelo. Los dispositivos del Internet de las Cosas, las flotas de robots y las aplicaciones de mensajería móvil generan flujos continuos de telemetría, eventos y comandos a alta frecuencia y en ambos sentidos, desde equipos de recursos acotados que operan sobre redes celulares o inalámbricas inestables y detrás de firewalls fuera del control de quien administra el servidor. Esta combinación plantea dos limitaciones de fondo para la ingeniería de redes: el costo, en ancho de banda y energía, de consultar periódicamente y de codificar los datos en formatos de texto; y la imposibilidad de que un servidor inicie comunicaciones hacia dispositivos que no admiten conexiones entrantes.

Frente a este panorama, muchos de estos sistemas han recurrido a la mensajería de tipo publicador/suscriptor, que desacopla a los participantes mediante un intermediario. En este campo se inscribe la combinación de MQTT y Protocol Buffers, un stack pensado para la comunicación en tiempo real con ancho de banda limitado. MQTT es un protocolo liviano de transporte de mensajes, organizado en torno a un broker, concebido para entornos restringidos (como los de comunicación máquina a máquina y de Internet de las Cosas) en los que se necesita una huella de código reducida o el ancho de banda es escaso. Protocol Buffers es un mecanismo, independiente del lenguaje y de la plataforma, para serializar datos estructurados y tipados en un formato más compacto y rápido que los textuales, a partir de un esquema compartido.

Su adopción en sistemas reales es amplia y heterogénea. En la mensajería a gran escala, Facebook recurrió a MQTT en Messenger para sostener una conexión persistente con sus servidores sin agotar la batería de los teléfonos, lo que permitió, en muchos casos, entregar los mensajes en cientos de milisegundos en lugar de varios segundos. En robótica, InOrbit, una empresa que ofrece una plataforma de orquestación de flotas de robots, implementa la comunicación entre servicios e integraciones desplegadas *on the edge* utilizando MQTT y mensajes definidos con Protocol Buffers, donde brokers desplegados en la nube reciben datos en tiempo real de miles de robots y dispositivos IoT, que a su vez son controlados por el mismo canal, aun en instalaciones con redes inestables. Desarrollado por la misma empresa, el producto de código abierto OpenRobOps libera las definiciones de su protocolo de comunicación en estructuras Protobuf que fijan el formato de las comunicaciones entre robot y plataforma.

Un rasgo de este esquema es especialmente relevante desde el encaminamiento: el dispositivo abre una conexión persistente hacia el broker alojado en la nube, cuya dirección suele estar habilitada en el firewall, y por esa misma conexión el broker le entrega los mensajes, sin necesidad de conexiones entrantes. El principio rector del stack es, entonces, trasladar la complejidad desde la red hacia el protocolo de aplicación: ningún participante necesita conocer la ubicación de los demás, sino solo la del broker, y el contrato sobre el formato de los datos se fija una única vez en un esquema distribuido entre los clientes. De este modo, la comunicación en tiempo real entre miles de entidades heterogéneas resulta viable sobre redes que no fueron concebidas para ese fin.

BORRADOR

1. Objetivos

Se desea realizar un análisis riguroso del stack de comunicación formado por MQTT y Protocol Buffers, como alternativa para el intercambio de datos en tiempo real, a alta frecuencia y con ancho de banda limitado, en aplicaciones de Internet de las Cosas, robótica y mensajería. El estudio sigue el recorrido de un mensaje desde el dispositivo que lo genera hasta el broker que lo distribuye, y evalúa cómo cada capa de la arquitectura contribuye a que ese recorrido sea eficiente y confiable.

Más que catalogar las prestaciones de ambas tecnologías, el trabajo busca explicar por qué resultan adecuadas para el problema planteado: sostener una comunicación continua sobre redes que el diseñador no controla, con enlaces inestables, firewalls de terceros y trayectos que atraviesan múltiples sistemas autónomos. Con ese fin, el análisis se articula en torno a dos ejes centrales de la materia, que pertenecen a capas distintas: la capa de aplicación (capa 7), donde operan MQTT y Protocol Buffers, y la capa de red (capa 3), donde el ruteo determina por qué camino llegan los mensajes al broker y cómo se restablece ese camino cuando la red cambia.

Para alcanzar este propósito se plantean los siguientes objetivos específicos:

- Presentar el modelo publicador/suscriptor y describir cómo lo implementa MQTT (broker, tópicos, niveles de calidad de servicio y sesiones), junto con el mecanismo de serialización de Protocol Buffers, y compararlos con alternativas como HTTP, CoAP y JSON.
- Analizar el funcionamiento de MQTT sobre la capa de transporte (TCP, keep-alive y TLS) y distinguir las garantías de entrega de sus niveles de calidad de servicio de la confiabilidad que ya ofrece TCP.
- Analizar el recorrido de los mensajes en la capa de red (capa 3): NAT y firewalls, y el efecto de la convergencia de los protocolos de ruteo interno (OSPF, con RIP y EIGRP como comparación) y externo (BGP) sobre las conexiones TCP de larga duración en las que se apoya MQTT.
- Contrastar el ruteo por destino IP de la capa de red con el ruteo por tópicos que realiza el broker en la capa de aplicación.
- Examinar casos de uso reales en robótica, mensajería e Internet de las Cosas, como los de InOrbit, para determinar las ventajas y limitaciones del stack en entornos de alta frecuencia y recursos acotados.
- Medir experimentalmente, sobre nodos emulados, el costo en bytes y latencia de MQTT y Protocol Buffers frente a HTTPS y JSON, y su comportamiento ante pérdidas, latencia y cortes de conexión.

2. Alcance

BORRADOR

Marco teórico

1. Modelo publicador/suscriptor

2. MQTT

3. Protocol Buffers

4. Stack MQTT y Protocol Buffers

5. Comparación con alternativas

5.1. MQTT, HTTP y CoAP

5.2. Protocol Buffers y JSON

BORRADOR

Desarrollo

1. MQTT sobre TCP y TLS

1.1. Conexión persistente y keep-alive

1.2. Cifrado con TLS

1.3. QoS MQTT sobre la confiabilidad de TCP

2. Capa de red

2.1. NAT y firewalls

2.2. Ruteo interno

2.3. Ruteo externo

2.4. Convergencia y conexiones de larga duración

3. Ruteo por tópicos y ruteo IP

4. Casos de uso

5. Diseño experimental

5.1. Plataforma común

Los dos experimentos corren sobre el mismo entorno, montado en una sola computadora, donde cada nodo es un ESP32 emulado con QEMU.

El ESP32 emulado no tiene WiFi. En su lugar, QEMU le provee una placa Ethernet emulada, que el firmware maneja como cualquier otra interfaz. Cada instancia usa la red de usuario de QEMU, que no requiere permisos de administrador: desde el nodo, la computadora anfitriona se ve en la dirección `10.0.2.2`. Ahí corre el broker Mosquitto, al que se conectan todos los nodos. Para varios nodos se lanzan varias instancias de QEMU en paralelo, cada una con su propia copia de la imagen de flash.

La emulación impone algunas restricciones. Los tiempos de ejecución no son los de una placa real, porque dependen de la carga de la computadora anfitriona. Además, la red de usuario no crea una interfaz en la computadora anfitriona, así que no se puede degradar el enlace de un nodo por separado. Para introducir latencia o pérdida en cada nodo hace falta conectar cada instancia a una interfaz virtual propia unida a un puente, y aplicar `netem` sobre esa interfaz.

BORRADOR

5.2. Experimento 1: MQTT contra HTTPS para telemetría

5.2.1. Planteo

5.2.2. Implementación

5.2.3. Resultados

5.2.4. Análisis de resultados

5.3. Experimento 2: sincronización de osciladores acoplados

5.3.1. Planteo

5.3.2. Implementación

5.3.3. Resultados

5.3.4. Análisis de resultados

BORRADOR

Conclusiones

BORRADOR

Bibliografía

[1] A. Banks, E. Briggs, K. Borgendale, y R. Gupta, Eds., «MQTT version 5.0», OASIS, OASIS Standard, 2019. [En línea]. Disponible en: https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html

[2] P. T. Eugster, P. A. Felber, R. Guerraoui, y A.-M. Kermarrec, «The many faces of publish/subscribe», *ACM Computing Surveys*, vol. 35, n.º 2, pp. 114-131, 2003, doi: 10.1145/857076.857078.

[3] Google, «Overview». Accedido: 19 de octubre de 2026. [En línea]. Disponible en: https://protobuf.dev/overview/

[4] L. Zhang, «Building Facebook Messenger». [En línea]. Disponible en: https://engineering.fb.com/2011/08/12/android/building-facebook-messenger/

[5] InOrbit, «InOrbit.AI: We make robots work». Accedido: 19 de octubre de 2026. [En línea]. Disponible en: https://www.inorbit.ai/

[6] inorbit-ai, «edge-sdk-python: InOrbit Python Edge SDK». Accedido: 19 de octubre de 2026. [En línea]. Disponible en: https://github.com/inorbit-ai/edge-sdk-python

[7] OpenRobOps, «OpenRobOps: Open source robot fleet management». Accedido: 19 de octubre de 2026. [En línea]. Disponible en: https://openrobops.org/

[8] OpenRobOps/oro, «oro: OpenRobOps open-source robot fleet management and operations platform». Accedido: 19 de octubre de 2026. [En línea]. Disponible en: https://github.com/OpenRobOps/oro

BORRADOR

Anexos

A. Mensajes del experimento de sincronización

BORRADOR